

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_3e301d0b-e59\agent-a197b80b9c93472e5.jsonl`  
- SHA-256: `cf6452c2af3c5f3360c1325551f3f0fcb95495bc1d26ed024c0b650d72b98294`  
- Source modified: `2026-06-13T18:45:38+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_3e301d0b-e59`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T18:45:07.538Z)

Map rally_gate.py exactly so we can wire Gate A into production.
Read: backend/pipeline/detectors/rally_gate.py and how backend/eval/distill_local.py + calibrate_local.py call it.
Report: signatures of gate_windows / apply_gate / count_net_crossings / estimate_play_axis; GatedWindow fields; the (start,end)-tuple-vs-rich-window-dict adaptation the caller must do; default min_crossings; and the exact, minimal way to apply rally_gate inside the live trajectory windowing behind a new min_crossings knob defaulting to 0 (=OFF, byte-identical to today). Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer (the indexer). Read the ACTUAL files and quote file:line. Be precise, not generic.

2. assistant (2026-06-13T18:45:09.184Z)

I'll map rally_gate.py and trace how it's called, then report the exact signatures and adaptation details.

3. user (2026-06-13T18:45:09.389Z)

```
1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.  
2  
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to  
4      the opponent for service after losing a point, that single trajectory looks like a  
5      rally. The discriminator is structural, not duration-based: a real rally has the  
6      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a  
7      single service feed crosses ~once.  
8  
9      This module is camera-agnostic and needs NO new model ? it works on the trajectory  
10     we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image  
11     axis with the larger shuttle spread) and the net position (median shuttle coord on  
12     that axis ? the shuttle clusters at the two ends and crosses the net region often,  
13     so the median sits near the net), then counts sign changes of (coord - net) across  
14     the visible points inside a candidate window, with a deadband to ignore jitter.  
15  
16     Gate A in the over-fire fix spectrum (B = player-stance state machine, future).  
17     Pure functions only ? no I/O, unit-tested; intended as a post-filter on the  
18     windows from trajectory_to_action_windows, or to fold into it later.  
19     """  
20     from __future__ import annotations  
21  
22     from dataclasses import dataclass  
23     from typing import List, Sequence, Tuple  
24  
25     Interval = Tuple[float, float]  
26  
27
```

```

28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06) ->

```

```

List[GatedWindow]:
93     """Annotate each (start,end) window with its shuttle net-crossing count and a
94     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95     axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96     axis, net, span = estimate_play_axis(points)
97     deadband = deadband_frac * span
98     out: List[GatedWindow] = []
99     for (s, e) in windows:
100         wp = _points_in(points, fps, s, e)
101         c = count_net_crossings(wp, axis, net, deadband)
102         vis = sum(1 for p in wp if p.visible)
103         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                               kept=c >= min_crossings))
105     return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

4. user (2026-06-13T18:45:09.947Z)

backend\eval\distill_local.py

5. user (2026-06-13T18:45:10.014Z)

backend\eval\calibrate_local.py

6. assistant (2026-06-13T18:45:11.350Z)

Now let me read both caller files:

7. user (2026-06-13T18:45:11.967Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10     filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11     active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12     -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21     python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse

```

```

27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37
38     Interval = Tuple[float, float]
39
40     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
41 "net_crossings"]
42     # Recall-oriented proposal: deliberately permissive so real rallies are among the
43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44     PROPOSAL = (0.005, 1.0, 0.5)
45     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
46
47     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48                         proposal: Tuple[float, float, float] = PROPOSAL) ->
49 Tuple[List[Interval], List[List[float]]]:
50     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
51 rows)."""
52     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
53     vt, mg, mwd = proposal
54     wins = TrackNetRunner.trajectory_to_action_windows(
55         points, fps=fps, frame_width=frame_width,
56         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
57     intervals = [(w["start"], w["end"]) for w in wins]
58     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
59 crossings
60     X: List[List[float]] = []
61     for w, g in zip(wins, gated):
62         dur = max(1e-6, w["end"] - w["start"])
63         win_frames = max(1.0, dur * fps)
64         X.append([
65             dur, # rally length (sec) ?
66             float(w["peak_velocity"]), # already normalized by
67             float(w["mean_velocity"]),
68             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
69             float(g.crossings), # net crossings (count) ?
70             the rally signal
71         ])
72     return intervals, X
73
74     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
75 List[Interval]:
76     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
77     vt, mg, mwd = BASELINE_LOCAL
78     wins = TrackNetRunner.trajectory_to_action_windows(
79         points, fps=fps, frame_width=frame_width,
80         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
81     return [(w["start"], w["end"]) for w in wins]
82
83     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
84         fps, fw = float(spec["fps"]), float(spec["frame_width"])
85         intervals, X = build_candidates(spec["trajectory"], fps, fw)

```

```

83     gts = load_golden_gts(spec["labels"])
84     y = label_candidates(intervals, gts, iou)
85     return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
86           "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}}
87
88
89     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
90                       threshold: float = 0.5) -> List[Interval]:
91         if not intervals:
92             return []
93         proba = model.predict_proba(X)
94         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
95         return merge_overlaps(pos, gap_tol=1.0)
96
97
98     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99           model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
111             clf, base = [], []
112             for i, held in enumerate(videos):
113                 others = [v for j, v in enumerate(videos) if j != i]
114                 X = [row for v in others for row in v["X"]]
115                 y = [t for v in others for t in v["y"]]
116                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118                 sc = score(preds, held["gts"], iou)
119                 bsc = score(held["baseline"], held["gts"], iou)
120                 clf.append(sc)
121                 base.append(bsc)
122                 print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
123                       f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124             def mean(xs, a):
125                 return sum(getattr(s, a) for s in xs) / len(xs)
126             print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
127                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
128                   f"F1={mean(base, 'f1'):.3f} R={mean(base, 'recall'):.3f}")
129             result["mean_learned_f1"] = mean(clf, "f1")
130             result["mean_baseline_f1"] = mean(base, "f1")
131
132         # Final model on ALL videos (for production / running on a fresh held-out video).
133         Xall = [row for v in videos for row in v["X"]]
134         yall = [t for v in videos for t in v["y"]]
135         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136         assert final.w is not None # set by fit()
137         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139         if model_out:
140             final.save(model_out)

```

```

141         print(f"final model (all videos) -> {model_out}")
142         result["model_out"] = model_out
143     return result
144
145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,
149                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161

```

8. user (2026-06-13T18:45:12.070Z)

```

1     """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2     on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3     100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5     Pipeline being tuned (all local, no Gemini at runtime):
6         WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7         -> rally_gate (min_crossings) -> rally windows
8
9     Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10    videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11    the honest "how precise is the local pipeline on an unseen video" number. The
12    output is a single local config + the precision it buys, with NO Gemini dependency
13    shipped.
14
15    Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16    calibration.{select_best, leave_one_video_out, improvement_report}.
17
18    Run:
19        python -m backend.eval.calibrate_local --manifest videos.json
20    where videos.json = [{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
21    (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22    """
23    from __future__ import annotations
24
25    import json
26    import argparse
27    from typing import Callable, Dict, List, Tuple
28
29    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30    from backend.pipeline.detectors import rally_gate
31    from backend.eval.calibrate_wasb import load_golden_gts
32    from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34    # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35    LocalConfig = Tuple[float, float, float, int]
36    BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0) # today's local default (windowing,
no gate)

```

```

37
38
39     def make_local_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], List]:
40         """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
41         (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
42         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
43
44         def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:
45             vt, mg, mwd, mc = cfg
46             windows = TrackNetRunner.trajectory_to_action_windows(
47                 points, fps=fps, frame_width=frame_width,
48                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
49             )
50             wins = [(w["start"], w["end"]) for w in windows]
51             if mc > 0:
52                 wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
53             return wins
54
55         return predict_fn
56
57
58     def local_grid() -> List[LocalConfig]:
59         return [(vt, mg, mwd, mc)
60                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
61                 for mg in (1.0, 2.0, 3.0)
62                 for mwd in (1.0, 2.0, 3.0)
63                 for mc in (0, 1, 2, 3)]
64
65
66     def build_videos(specs: List[Dict]) -> List[Dict]:
67         videos = []
68         for s in specs:
69             pf = make_local_predict_fn(s["trajectory"], float(s["fps"]),
float(s["frame_width"]))
70             gts = load_golden_gts(s["labels"])
71             if not gts:
72                 raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
73             videos.append({"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts":
gts})
74         return videos
75
76
77     def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
78         videos = build_videos(specs)
79         grid = local_grid()
80         print(f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
81               f"{len(grid)} configs, IoU={iou}, optimise {metric}) ===")
82
83         # Per-video: today's baseline vs the best-fit local config (optimistic ? fit on that
video).
84         per_video = []
85         for v in videos:
86             best_cfg, best_val = select_best(v["predict_fn"], grid, v["gts"], metric, iou)
87             imp = improvement_report(v["predict_fn"], BASELINE_LOCAL, best_cfg, v["gts"],
iou)
88             b, t = imp["baseline"], imp["tuned"]
89             print(f"\n[{v['name']}] {len(v['gts'])} Gemini rallies")
90             print(f"  baseline {BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f}
F1={b['f1']:.3f}")
91             print(f"  best-fit {best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f}
F1={t['f1']:.3f}")
92             f"(F1 {imp['f1_delta_pct']:+.0f}%, fit-on-self = optimistic)"
93             per_video.append({"name": v["name"], "best_cfg": best_cfg, "improvement": imp})
94

```

```

95     # Honest cross-video: pick config on the OTHER videos, score the held-out one.
96     lovo = leave_one_video_out(videos, grid, metric, iou)
97     # Baseline's held-out score (no tuning) for the honest before/after.
98     base_held = [evaluate(v["predict_fn"](BASELINE_LOCAL), v["gts"], iou) for v in
videos]
99     base_mean = sum(getattr(s, metric) for s in base_held) / len(base_held)
100
101     print("\n--- HONEST leave-one-video-out (the number to trust) ---")
102     if "error" in lovo:
103         print(f"    {lovo['error']} (need >=2 videos)")
104     else:
105         print(f"    baseline (untuned) mean held-out {metric}: {base_mean:.3f}")
106         print(f"    calibrated mean held-out {metric}:          {lovo['mean_held_out']:.3f}")
107         print(f"    f"± {lovo['std_held_out']:.3f} (gap {lovo['generalization_gap']:+.3f})")
108         for p in lovo["per_video"]:
109             print(f"        held-out {p['video']}: {p['held_out']:.3f} (cfg picked on
others: {p['config']})")
110     return {"per_video": per_video, "lovo": lovo, "baseline_held_out": base_mean,
"metric": metric}
111
112
113     def main() -> None:
114         ap = argparse.ArgumentParser(description="Calibrate the local-only pipeline against
Gemini silver labels.")
115         ap.add_argument("--manifest", required=True,
116                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
117         ap.add_argument("--iou", type=float, default=0.5)
118         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
119         args = ap.parse_args()
120         with open(args.manifest, encoding="utf-8") as f:
121             specs = json.load(f)
122         run(specs, iou=args.iou, metric=args.metric)
123
124
125     if __name__ == "__main__":
126         main()
127

```

9. assistant (2026-06-13T18:45:13.858Z)

Perfect. Now let me search for how trajectory_to_action_windows returns the "rich" window dict to understand the adaptation detail:

10. user (2026-06-13T18:45:14.713Z)

backend/pipeline/detectors/tracknet_runner.py: def trajectory_to_action_windows(

11. user (2026-06-13T18:45:15.964Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9     3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10    4. Convert that trajectory into high-motion "action windows" ? the same
11       candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12       pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.

```

```

15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"           # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                        # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                       # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                       # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p

```

```

72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl not found ? is this running on Windows with WSL2 installed?"
112         except subprocess.TimeoutExpired:
113             return False, "WSL healthcheck timed out."
114         out = (res.stdout or "").strip()
115         if res.returncode != 0:
116             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
117         return True, out
118
119 # ----- inference
120
121 def run_predict(self, video_win_path: str, output_win_dir: str,
122                 video_id: Optional[str] = None) -> Optional[str]:
123     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
124     None.
125
126     ``output_win_dir`` is a Windows directory (under the repo's output/) that
127     WSL writes into via its /mnt view, so the Windows process can read the CSV
128     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
129     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
130     a filename cannot collide on the output CSV.
131     """
132     if not self.cfg.weights_path:
133         logger.error(
134             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
135             "in config.json to the WSL path of your trained TrackNetV4 weights."

```

```

134         )
135         return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}_{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         # "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:
193         logger.error("TrackNet inference timed out after %ss.",
194                     self.cfg.timeout_sec)
195         return None
196
197     if res.returncode != 0:
198         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or

```

```

res.stdout)[-2000:])
197         return None
198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)

```

12. user (2026-06-13T18:45:17.808Z)

```

200         logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201         return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         dest = f"{self.cfg.wsl_stage_dir}/{base}"
219         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
220         try:
221             res = self._wsl_bash(cmd)
222         except subprocess.TimeoutExpired:
223             logger.error("Staging video into WSL timed out.")
224             return None
225         if res.returncode != 0 or "OK" not in (res.stdout or ""):
226             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
227             return None
228             return dest
229
230         # ----- CSV -> windows
231
232     @staticmethod
233     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235         points: List[TrajectoryPoint] = []
236         with open(csv_win_path, newline="") as f:
237             reader = csv.DictReader(f)
238             for row in reader:
239                 try:
240                     points.append(TrajectoryPoint(
241                         frame=int(row["Frame"]),
242                         visible=int(row["Visibility"]) == 1,
243                         x=float(row["X"]),
244                         y=float(row["Y"]),
245                     ))
246                 except (KeyError, ValueError):
247                     continue
248             points.sort(key=lambda p: p.frame)
249             return points
250
251     @staticmethod

```

```

252 def trajectory_to_action_windows(
253     points: List[TrajectoryPoint],
254     fps: float,
255     frame_width: float,
256     chunk_start: float = 0.0,
257     velocity_thresh: float = 0.02,
258     merge_gap: float = 2.0,
259     min_window_duration: float = 2.0,
260     chunk_index: Optional[int] = None,
261 ) -> List[Dict[str, Any]]:
262     """Convert a shuttle trajectory into candidate rally windows.
263
264     A frame is "active" when the shuttle is visible AND its inter-frame
265     displacement (normalised by frame width, per second) exceeds
266     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267     ``merge_gap`` seconds of each other are grouped into one window; short
268     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269     shape feeding the AI-handoff phase is identical.
270     """
271     if fps <= 0:
272         fps = 30.0
273     if frame_width <= 0:
274         frame_width = 1280.0
275
276     # Compute per-frame normalised velocity from the last *visible* point.
277     active = [] # (frame_idx, velocity)
278     prev: Optional[TrajectoryPoint] = None
279     for p in points:
280         if not p.visible:
281             prev = None # break the trajectory; absent shuttle = not in flight
282             continue
283         if prev is not None:
284             dframes = p.frame - prev.frame
285             if dframes > 0:
286                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                 velocity = (dist / frame_width) * (fps / dframes)
288                 if velocity > velocity_thresh:
289                     active.append((p.frame, velocity))
290             prev = p
291
292     if not active:
293         return []
294
295     max_gap_frames = int(merge_gap * fps)
296
297     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298         vels = [v for _, v in group]
299         return {
300             "start": group[0][0] / fps + chunk_start,
301             "end": group[-1][0] / fps + chunk_start,
302             "active_frame_count": len(group),
303             "peak_velocity": max(vels),
304             "mean_velocity": sum(vels) / len(vels),
305             "track_id_count": 1, # single shuttle; kept for window-shape parity
306             "chunk_index": chunk_index,
307         }
308
309     windows = []
310     group = [active[0]]
311     for af in active[1:]:
312         if af[0] - group[-1][0] <= max_gap_frames:
313             group.append(af)
314         else:
315             windows.append(_finalize(group))
316             group = [af]

```

```
317         windows.append(_finalize(group))
318
319         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320
```

13. assistant (2026-06-13T18:45:19.376Z)

Perfect. Now I have all the information. Let me compile the exact report.

14. user (2026-06-13T18:45:35.661Z)

Structured output provided successfully

15. assistant (2026-06-13T18:45:38.088Z)

Done. I've mapped rally_gate.py exactly with all signatures, GatedWindow fields, the tuple extraction pattern the caller must do, and the minimal wiring for production with min_crossings defaulting to 0 (OFF, byte-identical to today). The key detail: trajectory_to_action_windows returns rich dicts with start/end in seconds?extract as tuples `(w["start"], w["end"])` before passing to apply_gate.